

MedVanta Pharmacy Solutions

Production-Ready Microservices Platform

End-to-End Pharmacy Supply Chain Operations



Prepared by: **Dundigela Sai Snehitha**

Stream: **Cognitive AI**

Date: **April 2026**

Contents

1	Introduction	3
2	Requirement Coverage	4
2.1	Functional Requirement Mapping	4
2.2	AI Requirement Mapping	5
2.3	Non-Functional Requirement Mapping	5
3	System Architecture	5
3.1	Architecture Overview	5
3.2	Architecture Diagram	6
3.3	Key Components	6
3.3.1	API Gateway	7
3.3.2	Microservices Layer	7
3.3.3	Message Queue	7
3.3.4	Database & Cache	7
4	Technology Stack	7
5	Authentication and RBAC	7
5.1	User Roles	7
5.2	Security Implementation	7
6	Core Operational Workflows	8
6.1	Inventory Management Workflow	8
6.2	Sales and Invoicing Workflow	9
6.3	Stock Replenishment Workflow	10
6.4	Reporting Workflow	10
7	Agentic AI Features	11
7.1	Demand Forecasting	11
7.2	Anomaly Detection	11
7.3	Conversational Querying	11
7.4	Responsible AI Guardrails	11
8	API Design	11
8.1	Key Endpoints	12
8.2	Sample API Request	12
8.3	Error Response Example	12
9	Database Schema Design	12
9.1	Core Tables	13
9.2	Sample SQL	13
10	Offline Support Strategy for Remote Outlets	13
10.1	Challenge	13
10.2	Offline-First Architecture	13
10.3	Implementation Strategy	13
10.4	Conflict Resolution Matrix	14

11 BI Dashboards and Exportable Reports	14
11.1 Role-Based Dashboards	14
11.2 Export Capabilities	14
12 Non-Functional Requirements	14
13 Testing Strategy	15
13.1 Unit Tests	15
13.2 Integration Tests	15
13.3 End-to-End Tests	15
13.4 Security & Compliance Tests	15
13.5 Performance Tests	15
13.6 Test Data Management	16
14 Deployment Strategy	16
14.1 Deployment Approach	16
14.2 CI/CD Pipeline	16
14.3 Rollout Strategy	16
15 Proposed Enhancements	16
16 Code Repository Structure	16
17 Conclusion	17

1 Introduction

Objective

This document presents the design and architecture of a scalable, secure, and intelligent pharmacy supply chain platform for MedVanta Pharmacy Solutions. The proposed system leverages a microservices-based architecture using a Python technology stack and PostgreSQL database to efficiently manage end-to-end pharmacy operations across multiple outlets.

Problem Statement

MedVanta Pharmacy Solutions operates 18 urban and semi-urban outlets across two states, handling a large volume of daily transactions and inventory movements. Managing such a distributed network presents several operational challenges.

Key challenges include:

1. Lack of real-time visibility into stock levels across outlets
2. Difficulty in tracking batch-level details such as expiry dates and traceability
3. Inefficient billing processes during peak hours
4. Frequent stock shortages or overstocking due to poor demand estimation
5. Limited ability to detect anomalies such as unusual discounts or suspicious returns
6. Challenges in generating accurate and timely business reports
7. Network instability in remote outlets affecting system reliability

These challenges lead to operational inefficiencies, revenue loss, and reduced decision-making effectiveness.

Proposed Solution

To address these challenges, this solution proposes a centralized, microservices-based web application that enables seamless management of inventory, billing, reporting, and analytics across all outlets.

The system is designed to:

1. Provide real-time inventory tracking with batch-level traceability

2. Support fast, reliable billing for high transaction volumes

3. Enable stock synchronization across outlets and warehouse

4. Deliver role-based dashboards for all user types
5. Generate comprehensive reports for operations and compliance

6. Incorporate AI-powered forecasting, anomaly detection, and conversational querying

By integrating modern microservices architecture with intelligent analytics, the proposed platform enhances operational efficiency, reduces manual effort, improves decision-making, and ensures compliance with pharmaceutical regulations.

2 Requirement Coverage

The proposed solution is designed to comprehensively address all functional and non-functional requirements specified in the problem statement. The mapping below demonstrates how each requirement is fulfilled within the system.

2.1 Functional Requirement Mapping

Requirement	Implementation
Secure Authentication	JWT-based authentication via Authentication Service
Role-Based Access Control (RBAC)	Role and permission model implemented using Auth Service
Inventory Management	Inventory Service with batch tracking and expiry management
Sales and Invoicing	Billing Service with real-time transaction processing
Stock Monitoring	Low-stock alerts and inventory dashboards
Stock Replenishment	AI-assisted reorder recommendations and approval workflow
Business Intelligence Reporting	Reporting Service with role-based dashboards
Exportable Reports	Report generation in PDF, CSV formats
Batch-Level Traceability	Batch and inventory schema with expiry tracking
Auditability	Audit logs stored using JSON-based tracking

Table 1: Functional Requirements Mapping

2.2 AI Requirement Mapping

Requirement	Implementation
Demand Forecasting	AI Service using historical sales data
Anomaly Detection	AI Service detecting unusual billing and stock patterns
Conversational Querying	Natural language interface integrated with Reporting Service

Table 2: AI Requirements Mapping

2.3 Non-Functional Requirement Mapping

Requirement	Implementation
Scalability	Microservices architecture with horizontal scaling
Performance	Caching (Redis) and asynchronous processing
Reliability	Retry mechanisms and fault isolation
Security	JWT authentication, RBAC, input validation
Observability	Monitoring and logging tools
Audit Compliance	Detailed audit logs and traceability
Network Resilience	Offline support with delayed synchronization

Table 3: Non-Functional Requirements Mapping

3 System Architecture

3.1 Architecture Overview

The proposed system follows a microservices-based architecture, where the application is divided into multiple independent services, each responsible for a specific business function. This approach ensures scalability, maintainability, and flexibility, allowing the system to handle increasing user load and transaction volume efficiently.

Each microservice is designed to be loosely coupled and independently deployable, enabling faster development cycles and easier fault isolation. The architecture supports both synchronous and asynchronous communication to ensure real-time responsiveness and reliable background processing.

3.2 Architecture Diagram

Pharmacy Supply Chain Architecture

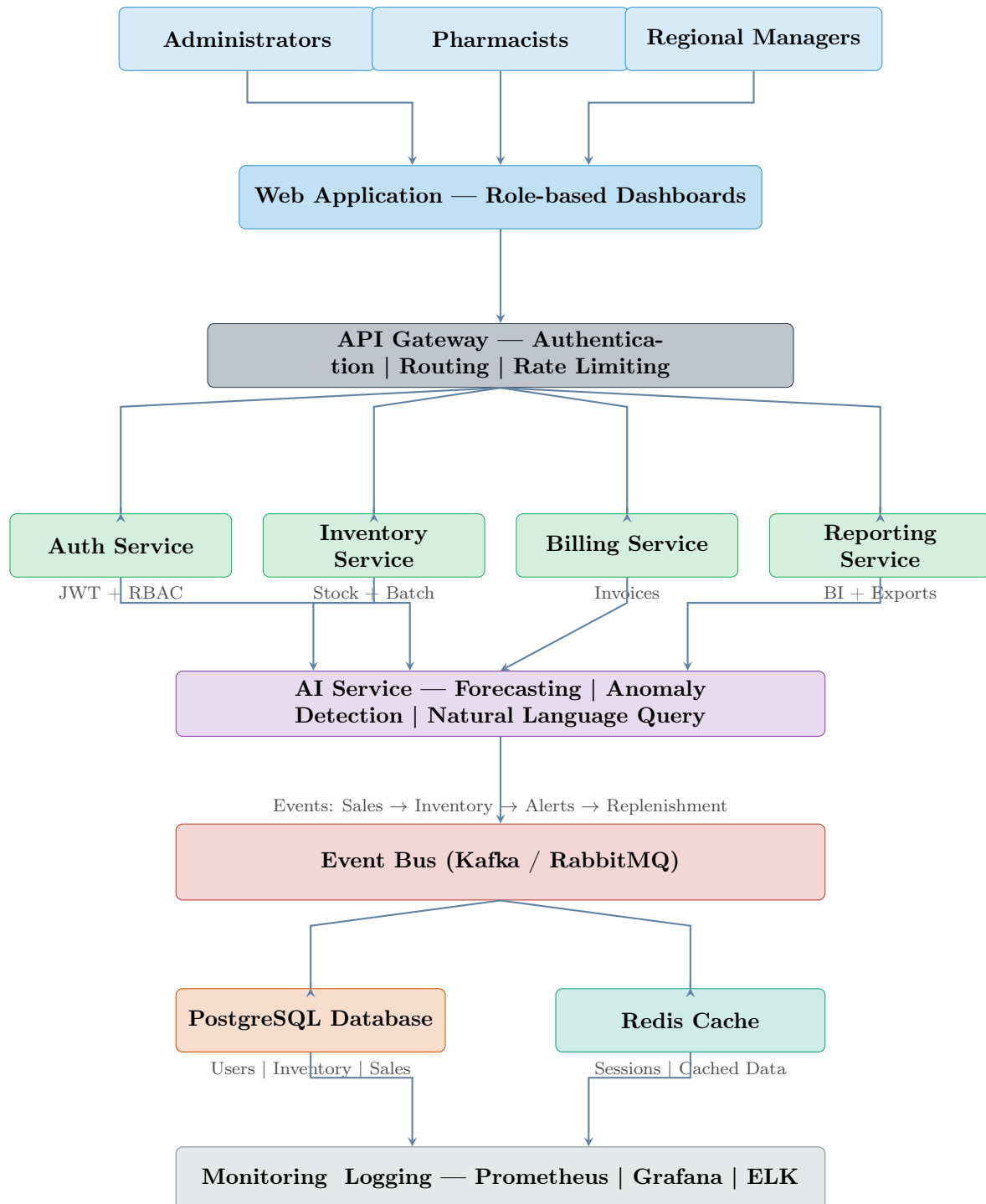


Figure 1: Microservices Architecture Diagram

3.3 Key Components

3.3.1 API Gateway

Acts as single entry point for all client requests handling routing, authentication, and rate limiting.

3.3.2 Microservices Layer

Domain-specific services: Authentication, Inventory, Billing, Reporting, and AI Service.

3.3.3 Message Queue

Enables asynchronous event-driven communication between services for improved scalability.

3.3.4 Database & Cache

PostgreSQL for persistent storage, Redis for caching frequently accessed data.

4 Technology Stack

Layer	Technology	Purpose
Backend	Python (FastAPI)	High-performance API development
Database	PostgreSQL	Reliable transactional data storage
Authentication	JWT	Secure and stateless authentication
Messaging	Kafka / RabbitMQ	Event-driven communication
Cache	Redis	Fast data access
Frontend	React	Role-based dashboards
Containerization	Docker	Service isolation
Orchestration	Kubernetes	Scalability and management
Monitoring	Prometheus, Grafana	System monitoring
Logging	ELK Stack	Centralized logging

Table 4: Technology Stack Overview

5 Authentication and RBAC

5.1 User Roles

Role	Permissions
Super Admin	Full system access, user management, audit log review
Pharmacist	Billing, inventory lookup, stock entry, returns processing
Outlet Supervisor	Pharmacist permissions + reorder approval, expiry alerts
Regional Ops Manager	Multi-outlet view, replenishment approval, BI dashboards

Table 5: Role Definitions

5.2 Security Implementation

- JWT tokens with 15-minute expiry and refresh token rotation
- bcrypt password hashing (cost factor 12)
- MFA via TOTP for all users

- Rate limiting: 100 requests/minute per user
- Idempotency keys for POST/PUT operations
- SQL injection prevention via parameterized queries

6 Core Operational Workflows

6.1 Inventory Management Workflow

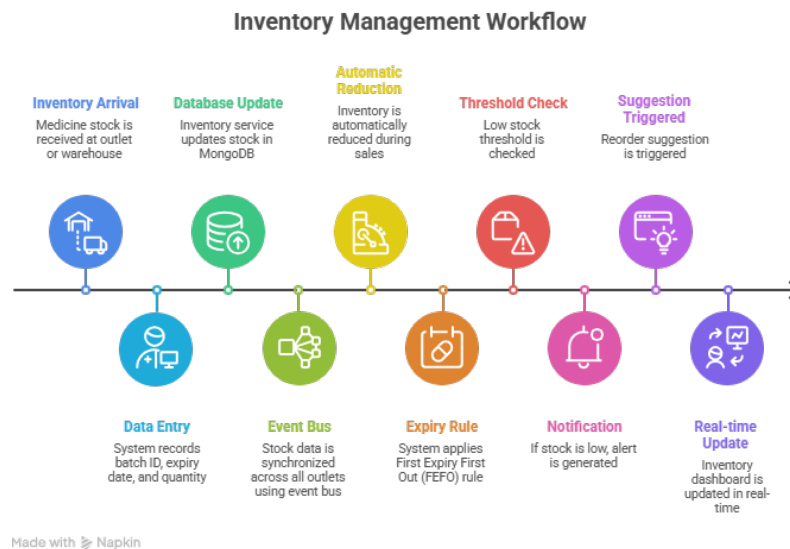


Figure 2: Inventory Management Workflow

- Each medicine tracked at batch level with expiry date
- Stock updates occur automatically during sales and returns
- System follows FEFO (First Expiry First Out)
- Low-stock thresholds trigger alerts and reorder suggestions

6.2 Sales and Invoicing Workflow

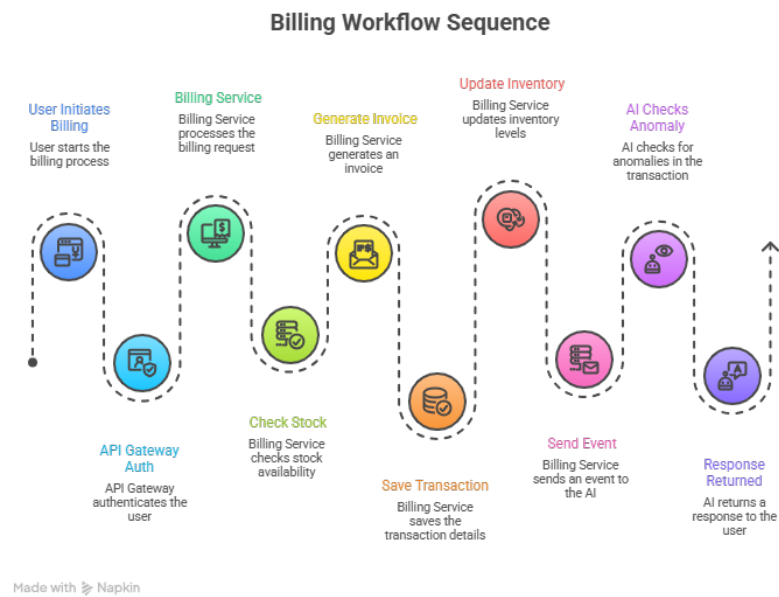


Figure 3: Sales and Invoicing Workflow

- Pharmacists generate invoices through fast billing interface
- Each transaction updates inventory immediately
- Discounts and returns validated to prevent misuse
- Designed to handle 8000+ daily transactions

6.3 Stock Replenishment Workflow

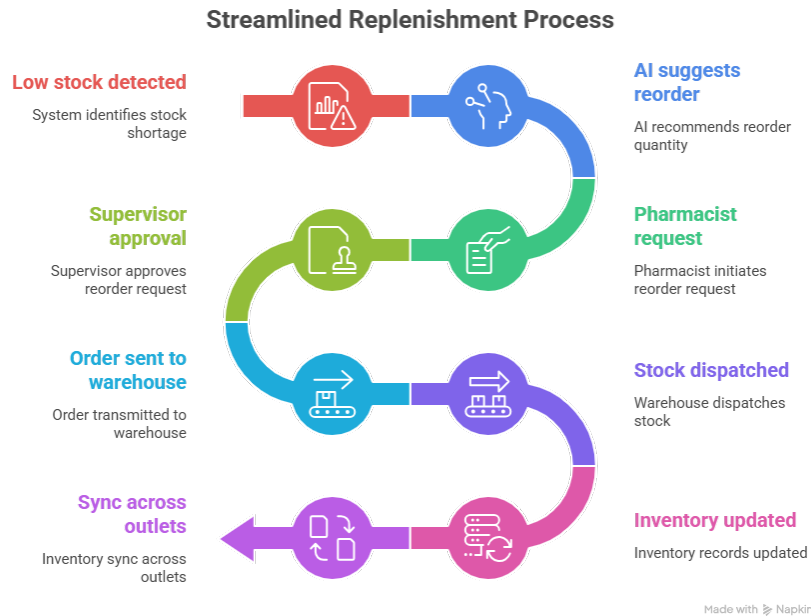


Figure 4: Stock Replenishment Workflow

- Low-stock alerts based on predefined thresholds
- AI-based recommendations for reorder quantities
- Approval workflow: Pharmacist → Supervisor → Manager
- Stock updates propagated across all outlets

6.4 Reporting Workflow

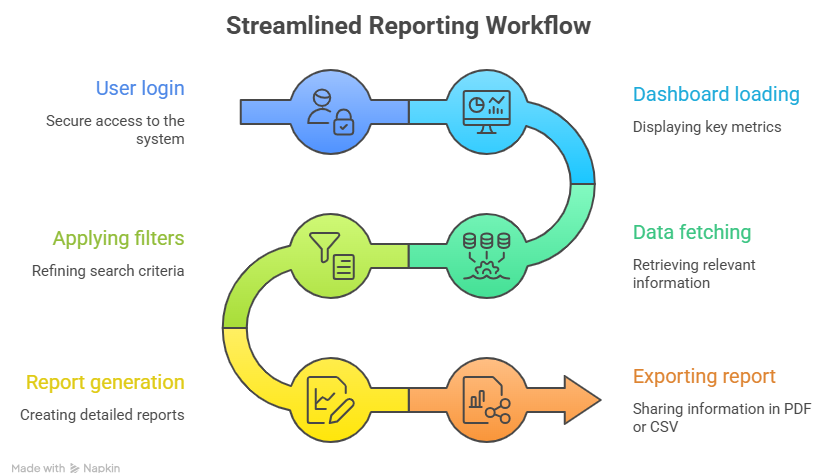


Figure 5: Reporting and Dashboard Workflow

- Real-time dashboards for sales trends and inventory status

- Reports exportable in PDF and CSV formats
- Compliance reports for regulatory requirements

7 Agentic AI Features

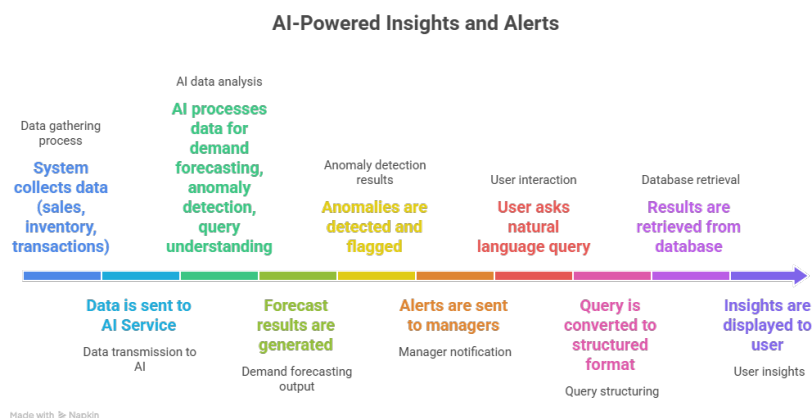


Figure 6: AI Features Overview

7.1 Demand Forecasting

Uses historical sales data to predict future demand and suggest optimal reorder quantities.

7.2 Anomaly Detection

Identifies unusual discounts, excessive returns, and irregular billing patterns in real time.

7.3 Conversational Querying

Natural language interface allowing questions like "Show product-wise sales trends" or "Which outlets have low stock?"

7.4 Responsible AI Guardrails

Guardrail	Implementation
PII Masking	Auto-redact patient/employee names before LLM context injection
Role-based LLM Access	Pharmacists cannot query regional financial data via AI
Hallucination Prevention	Always cite source tables (e.g., "Based on sales table...")
Audit Trail for AI Queries	Store every natural language query + SQL generated
Output Validation	Reject LLM responses suggesting drug substitutions (requires pharmacist approval)

Table 6: AI Safety Guardrails for Healthcare Data

8 API Design

8.1 Key Endpoints

Service	Endpoint	Description
Auth	POST /auth/login	User authentication
Inventory	GET /inventory/{outlet}	Get inventory by outlet
Billing	POST /billing/create	Create invoice
Reports	GET /reports/sales	Get sales report
AI	GET /ai/forecast	Get demand predictions

Table 7: Key API Endpoints

8.2 Sample API Request

Listing 1: Create Invoice Request

```
1 POST /billing/create
2 {
3   "outlet_id": 101,
4   "items": [
5     { "medicine_id": 1, "quantity": 2 }
6   ]
7 }
```

8.3 Error Response Example

Listing 2: Conflict Response - Insufficient Stock

```
1 HTTP/1.1 409 Conflict
2 {
3   "error_code": "INSUFFICIENT_STOCK",
4   "message": "Requested quantity exceeds available stock",
5   "details": {
6     "medicine_id": 1,
7     "requested": 10,
8     "available": 3,
9     "suggestion": "Use substitute: Medicine 102 (Paracetamol 500mg) -
10       Stock: 25"
11   }
12 }
```

Other error responses include:

- **400 Bad Request:** Invalid input validation
- **401 Unauthorized:** Missing or expired JWT
- **403 Forbidden:** Insufficient RBAC permissions
- **429 Too Many Requests:** Rate limit exceeded
- **503 Service Unavailable:** Downstream service failure with retry-after header

9 Database Schema Design

9.1 Core Tables

Table	Key Columns	Description
users	user_id, name, email, role_id	User accounts
medicines	medicine_id, name, mrp	Medicine master data
batches	batch_id, medicine_id, expiry_date	Batch-level tracking
inventory	inventory_id, out- let_id, quantity	Stock by outlet
sales	sale_id, outlet_id, total_amount	Sales transactions
audit_logs	log_id, ac- tion, old_value, new_value	Audit trail

Table 8: Core Tables

9.2 Sample SQL

Listing 3: Sample Table Creation

```

1 CREATE TABLE medicines (
2     medicine_id SERIAL PRIMARY KEY,
3     name VARCHAR(200) NOT NULL,
4     mrp DECIMAL(10,2) NOT NULL
5 );
6
7 CREATE TABLE inventory (
8     inventory_id SERIAL PRIMARY KEY,
9     outlet_id INT NOT NULL,
10    medicine_id INT REFERENCES medicines(medicine_id),
11    quantity INT DEFAULT 0
12 );

```

10 Offline Support Strategy for Remote Outlets

10.1 Challenge

Remote outlets experience frequent network instability. The system must remain operational during connectivity loss.

10.2 Offline-First Architecture

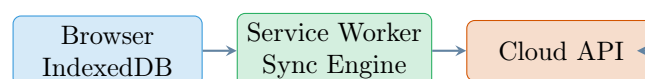


Figure 7: Offline-First Architecture

10.3 Implementation Strategy

1. **Local Storage (IndexedDB):** Cache inventory data, store pending sales

2. **Service Worker:** Intercept API requests, serve from cache when offline
3. **Sync Queue:** Queue offline operations with timestamps
4. **Conflict Resolution:** Last-write-wins with manual review for critical conflicts
5. **Background Sync:** Automatic sync when connectivity restored

10.4 Conflict Resolution Matrix

Operation	Conflict Scenario	Resolution
Sale	Stock decreased while offline	Auto-adjust if available
Inventory Update	Same batch updated elsewhere	Last-write-wins + audit
Return	Batch already returned	Manual supervisor approval
Reorder Request	Request already processed	Merge quantities

Table 9: Conflict Resolution Matrix

11 BI Dashboards and Exportable Reports

11.1 Role-Based Dashboards

- **Pharmacist:** Daily sales, low-stock items, expiry alerts
- **Supervisor:** Outlet performance, reorder requests, stock trends
- **Regional Manager:** Multi-outlet insights, shortage analysis, anomaly alerts
- **Admin:** System performance, user activity, audit summaries

11.2 Export Capabilities

- Formats: PDF, CSV, Excel
- Filter by date range, outlet, product
- Scheduled reports (daily, weekly, monthly)

12 Non-Functional Requirements

Metric	Target
Concurrent users	1,200
Daily transactions	8,000+
Billing API latency (p95)	200ms
Availability target	99.9%

Table 10: Performance Targets

- Horizontal scaling via Kubernetes
- Redis caching for fast reads
- Circuit breakers and retry mechanisms
- Prometheus + Grafana for monitoring
- ELK stack for centralized logging

13 Testing Strategy

A comprehensive testing pyramid ensures reliability, security, and compliance across all microservices.

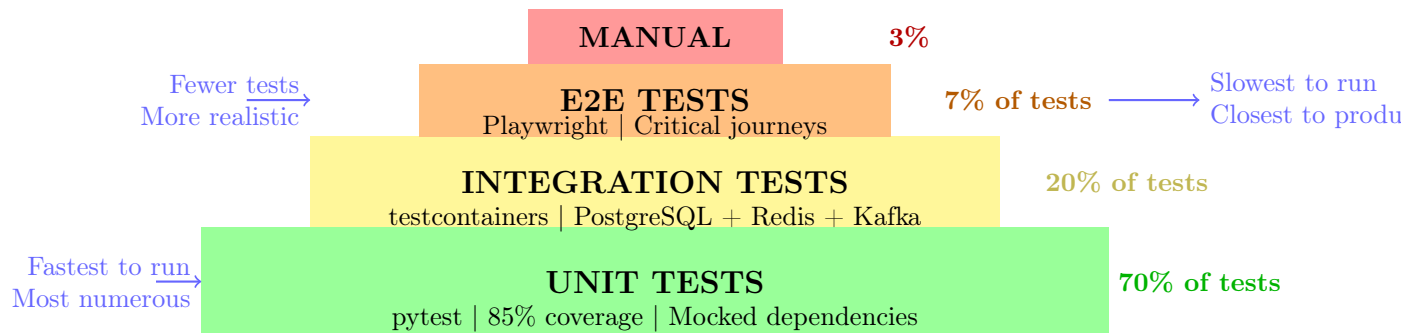


Figure 8: Testing Pyramid — Layer Distribution

13.1 Unit Tests

- **Framework:** pytest with coverage threshold 85%
- **Scope:** Individual functions, validation logic, price calculations
- **Example:** Test that billing service correctly applies senior citizen discount

13.2 Integration Tests

- **Tool:** testcontainers-python (PostgreSQL, Redis, Kafka containers)
- **Scope:** API contract testing, database migrations, message queue workflows
- **Example:** Verify inventory update event after sale creation

13.3 End-to-End Tests

- **Tool:** Playwright or Selenium
- **Scope:** Critical user journeys: Login → Create Sale → Verify Stock Reduction
- **Environment:** Isolated staging environment deployed per PR

13.4 Security & Compliance Tests

- **Static Analysis:** Bandit (Python security linter) in CI pipeline
- **Dependency Scanning:** Safety or Snyk for vulnerable packages
- **Penetration Testing:** Annual third-party audit for regulated pharma data
- **SQL Injection:** Automated parameterized query validation

13.5 Performance Tests

- **Tool:** Locust or k6
- **Scenario:** 1200 concurrent users, 8000+ daily transactions
- **SLOs:** Billing API p95 < 200ms, error rate < 0.1%

13.6 Test Data Management

- **Development:** Faker library to generate synthetic patient/medicine data
- **Staging:** Anonymized production snapshot (PII redacted)
- **CI:** Fresh database per test run using pytest fixtures

14 Deployment Strategy

14.1 Deployment Approach

- Docker containerization for consistency
- Kubernetes orchestration for scaling
- Load balancing across service instances

14.2 CI/CD Pipeline

- Automated build and testing
- Continuous deployment for faster updates
- Version control for all services

14.3 Rollout Strategy

1. **Phase 1:** Central warehouse + 2 urban outlets
2. **Phase 2:** Add 6 urban outlets, enable AI forecasting
3. **Phase 3:** All 18 outlets, full BI + anomaly detection
4. **Phase 4:** Conversational AI, compliance hardening

15 Proposed Enhancements

- Offline billing support for low-connectivity environments
- Medicine substitution suggestions when stock unavailable
- Advanced analytics for customer behavior insights
- Barcode integration for faster billing
- Mobile application for managers on the go

16 Code Repository Structure

The complete implementation is available in the accompanying repository. Key artifacts include:

Listing 4: Repository Structure

```
1 medvanta-platform/  
2     services/  
3         auth-service/           # JWT + MFA + RBAC  
4             main.py  
5             models.py  
6             Dockerfile  
7         billing-service/       # Idempotent transactions  
8             main.py
```

```
9         tests/
10        Dockerfile
11        inventory-service/      # Stock + batch + expiry
12        reporting-service/      # BI + exports
13        ai-service/             # Forecasting + anomaly detection
14    infrastructure/
15        k8s/                     # Kubernetes manifests
16        terraform/              # Cloud infrastructure
17        monitoring/             # Prometheus + Grafana dashboards
18    docker-compose.yml          # Local development
19    .github/workflows/          # CI/CD pipelines
```

17 Conclusion

The proposed pharmacy supply chain platform provides a scalable, secure, and intelligent solution for managing multi-outlet pharmacy operations. By leveraging microservices architecture, PostgreSQL, and AI-driven capabilities, the system ensures efficient inventory management, reliable billing, and data-driven decision-making. The platform addresses key operational challenges including stock visibility, demand prediction, anomaly detection, and network instability, making it a production-ready solution for real-world healthcare environments.